

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчёт

Лабораторная работа №3

Выполнил:

Карпов Данил Иванович

Группа

J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Миграция ранее разработанного веб-приложения «Minion Courses» - платформы онлайн-обучения - на фреймворк Vue.js 3 с использованием Composition API. Целью работы является создание полноценного одностраничного приложения (SPA), настройка клиентской маршрутизации, централизованного управления состоянием через Pinia, использование Composables для выделения переиспользуемой логики и организация работы с внешним REST API посредством библиотеки Axios.

Ход работы

Процесс разработки включал следующие этапы:

- **Инициализация проекта:** Развёрнута структура SPA с помощью Vite и шаблона `vue-ts`. Установлены зависимости: Vue Router 4, Pinia, Axios, Bootstrap 5, Bootstrap Icons. Настроен псевдоним пути `@/` в `vite.config.ts` и `tsconfig.app.json` для удобной адресации файлов внутри `src/`.
- **Типизация (TypeScript):** Созданы интерфейсы для всех сущностей предметной области в файле `src/types/index.ts`: `User`, `Course`, `Enrollment`, `Certificate`, а также вспомогательные типы для полезных нагрузок запросов (`LoginPayload`, `RegisterPayload`).
- **Управление состоянием (Pinia):** Реализовано хранилище авторизации `src/stores/auth.ts` на Composition API (setup-style store). Хранилище управляет JWT-токеном и объектом текущего пользователя, предоставляет действия `login`, `register`, `logout` и `fetchUser`. Токен персистируется в `localStorage` вручную (см. Рис. 1).

- **Composable useTheme:** Логика переключения тёмной/светлой темы вынесена в `src/composables/useTheme.ts`.
Используется реактивная переменная `theme` уровня модуля и `watch` для синхронизации атрибута `data-bs-theme` на элементе `<html>` и `localStorage` (см. Рис. 2).
- **Composable useApi (Axios):** Взаимодействие с бэкендом централизовано в `src/composables/useApi.ts`. Настроен `axios`-инстанс с базовым URL `http://localhost:3001` и перехватчиком запроса, автоматически прикрепляющим JWT-токен в заголовок `Authorization: Bearer ...` (см. Рис. 3).
- **Composable useCourses:** Логика загрузки списка курсов и одиночного курса выделена в `src/composables/useCourses.ts`. Composable инкапсулирует реактивные состояния `loading` и `error`, что избавляет компоненты от дублирования шаблонного кода.
- **Маршрутизация (Vue Router):** Настроен Vue Router 4 в `src/router/index.ts` с ленивой загрузкой всех страниц через динамические импорты. Реализованы `navigation guards`: защищённые маршруты `/profile` и `/teacher` перенаправляют неавторизованного пользователя на страницу входа; маршрут `/teacher` дополнительно проверяет роль пользователя (см. Рис. 4).
- **Компонентная архитектура:** Приложение разделено на переиспользуемые компоненты: `AppNavbar.vue` (навигационная панель с реактивным состоянием авторизации и переключателем темы), `CourseCard.vue` (карточка курса, используется на главной и в каталоге), `SkeletonCard.vue` (skeleton-заглушка при загрузке).

```

// src/stores/auth.ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import type { User, LoginPayload, RegisterPayload } from '@/types'
import { useApi } from '@/composables/useApi'

export const useAuthStore = defineStore('auth', () => {
  const token = ref<string | null>(localStorage.getItem('token'))
  const user = ref<User | null>(null)

  const isLoggedIn = computed(() => !!token.value)

  const { get, post } = useApi()

  async function fetchUser() {
    if (!token.value) return
    try { user.value = await get<User>('/me') }
    catch { logout() }
  }

  async function login(payload: LoginPayload) {
    const data = await post<{ token: string; user: User }>('/login', payload)
    token.value = data.token
    user.value = data.user
    localStorage.setItem('token', data.token)
  }

  function logout() {
    token.value = null
    user.value = null
    localStorage.removeItem('token')
  }

  return { token, user, isLoggedIn, fetchUser, login, register, logout }
})

```

Рис. 1 - Pinia store авторизации (src/stores/auth.ts)

```

// src/composables/useTheme.ts
import { ref, watch } from 'vue'

const theme = ref<'light' | 'dark'>(
  (localStorage.getItem('theme') as 'light' | 'dark') || 'light'
)

watch(
  theme,
  (val) => {
    document.documentElement.setAttribute('data-bs-theme', val)
    localStorage.setItem('theme', val)
  },
  { immediate: true }
)

export function useTheme() {
  function toggleTheme() {
    theme.value = theme.value === 'light' ? 'dark' : 'light'
  }
  return { theme, toggleTheme }
}

```

Рис. 2 - Composable управления темой (src/composables/useTheme.ts)

```
// src/composables/useApi.ts
import axios from 'axios'

const BASE_URL = 'http://localhost:3001'

const client = axios.create({ baseURL: BASE_URL })

client.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

export function useApi() {
  async function get<T>(url: string, params?: Record<string, unknown>): Promise<T> {
    const { data } = await client.get<T>(url, { params })
    return data
  }

  async function post<T>(url: string, body: unknown): Promise<T> {
    const { data } = await client.post<T>(url, body)
    return data
  }

  return { get, post }
}
```

Рис. 3 - Composable для работы с API через Axios (src/composables/useApi.ts)


```

// src/router/index.ts
import { createRouter, createWebHistory } from 'vue-router'
import { useAuthStore } from '@stores/auth'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/', name: 'home', component: () => import('@views/homeView.vue') },
    { path: '/login', name: 'login', component: () => import('@views/loginView.vue') },
    { path: '/courses', name: 'courses', component: () => import('@views/coursesView.vue') },
    { path: '/courses/:id', name: 'course', component: () => import('@views/courseView.vue') },
    {
      path: '/profile', name: 'profile',
      component: () => import('@views/ProfileView.vue'),
      meta: { requiresAuth: true },
    },
    {
      path: '/teacher', name: 'teacher',
      component: () => import('@views/TeacherView.vue'),
      meta: { requiresAuth: true, requiresTeacher: true },
    },
  ],
})

router.beforeEach((to) => {
  const auth = useAuthStore()
  if (to.meta.requiresAuth && !auth.isLoggedIn) return { name: 'login' }
  if (to.meta.requiresTeacher && auth.user?.role !== 'teacher') return { name: 'login' }
})

export default router

```

Рис. 4 - Конфигурация Vue Router с navigation guards (src/router/index.ts)

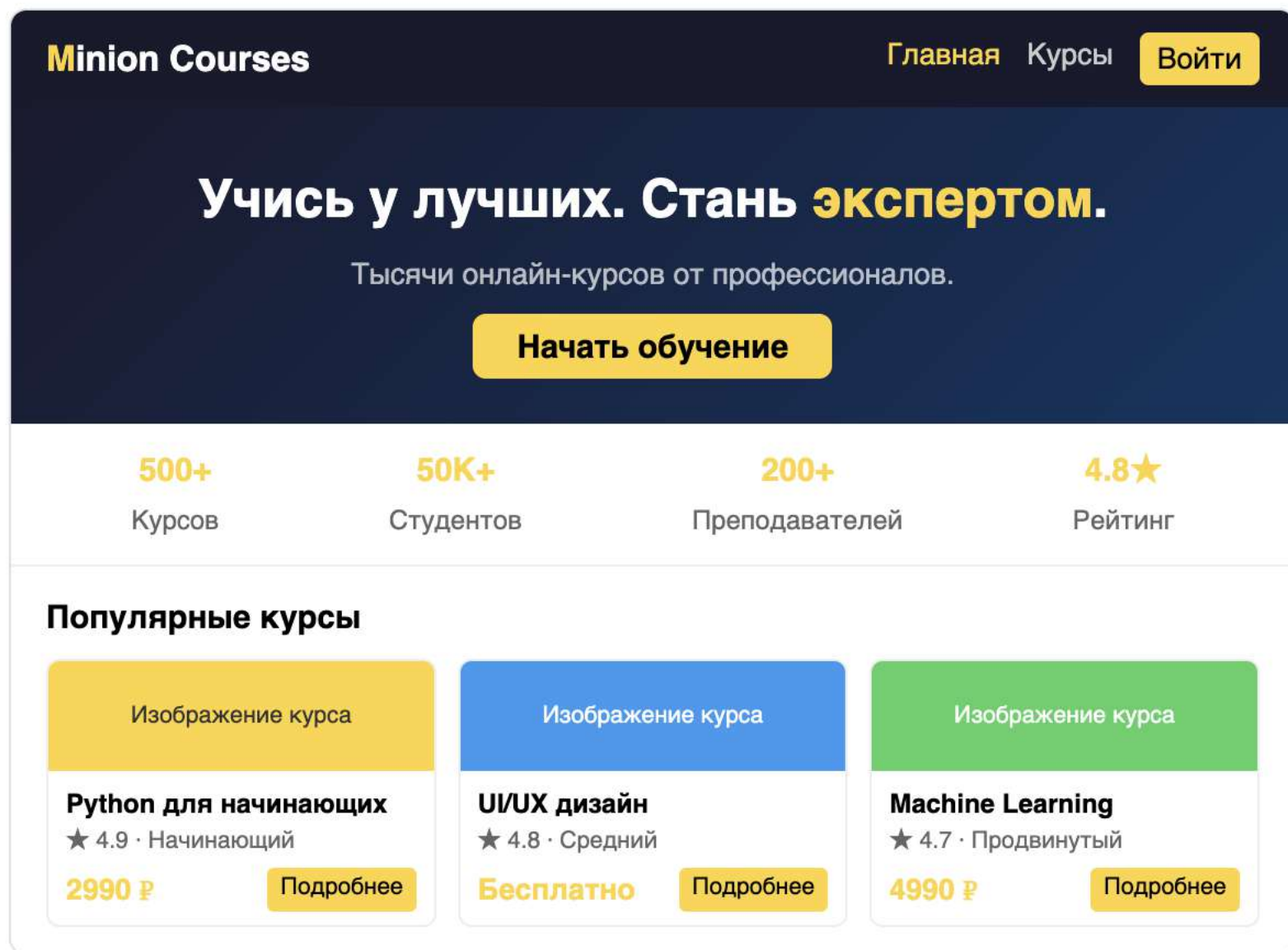


Рис. 5 - Главная страница приложения (HomeView)

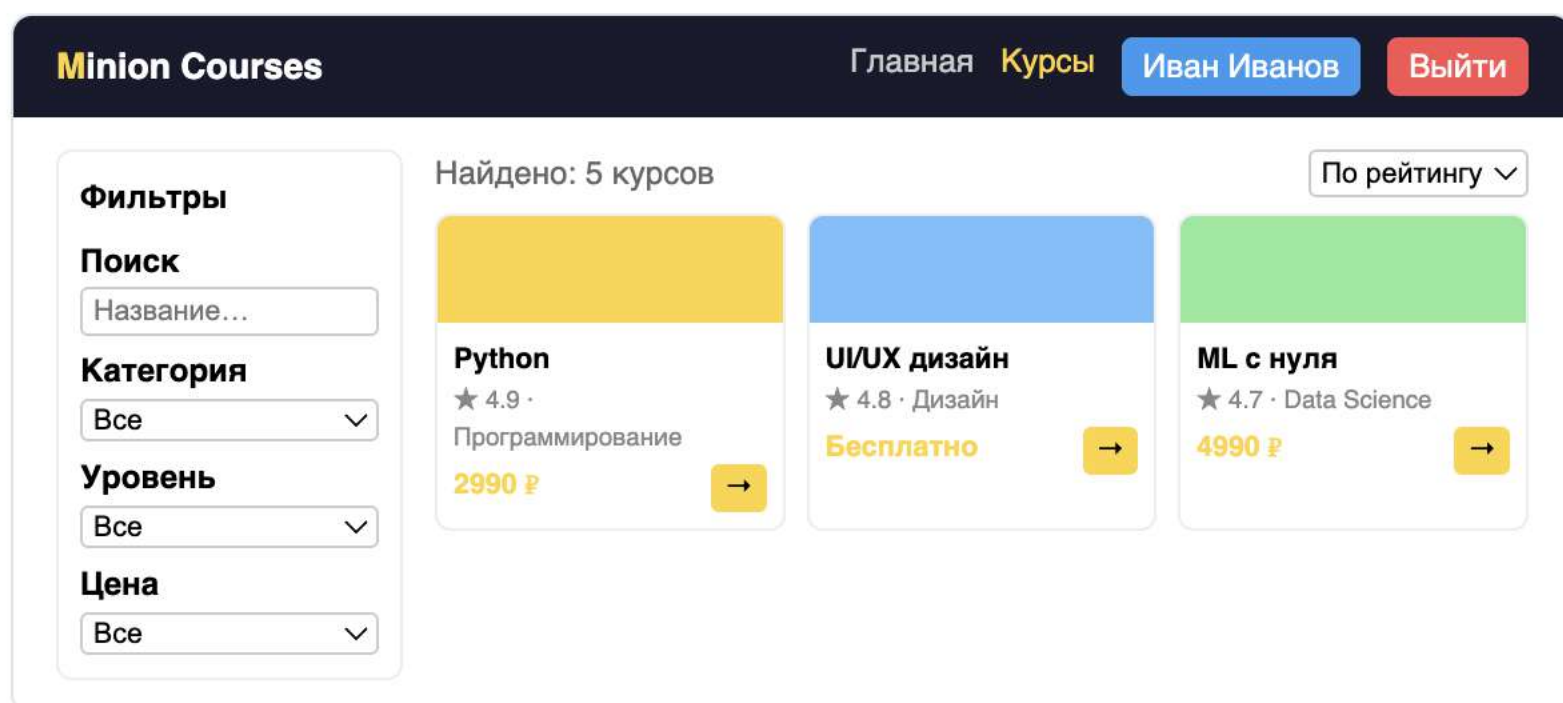


Рис. 6 - Каталог курсов с фильтрами (SearchView)

Вывод

В ходе лабораторной работы было выполнено полное переосмысление и миграция многостраничного HTML-приложения «Minion Courses» на современный фреймворк Vue.js 3 с использованием Composition API и TypeScript. Итоговое приложение представляет собой полноценное SPA с клиентской маршрутизацией через Vue Router 4.

Использование Pinia позволило централизовать управление состоянием авторизации и обеспечить реактивный доступ к данным текущего пользователя из любого компонента без передачи props через промежуточные уровни. Composables `useApi`, `useCourses` и `useTheme` позволили выделить повторяющуюся логику работы с API, загрузки данных и управления темой в переиспользуемые единицы кода.

Компонентная архитектура обеспечила чистое разделение ответственности: общие элементы интерфейса (`AppNavbar`, `CourseCard`, `SkeletonCard`) переиспользуются на нескольких страницах, а каждая страница инкапсулирует только свою уникальную логику. Navigation guards на уровне роутера надёжно защищают приватные маршруты без дублирования проверок в компонентах.

В результате проект стал значительно более структурированным, расширяемым и поддерживаемым по сравнению с исходным vanilla-решением. Полученный опыт работы с реактивностью Vue 3, Composition API и экосистемой Vite является ключевой основой для разработки профессиональных фронтенд-приложений.